3. Interpolation

3.1. Motivation: Why Interpolation Matters in Physics

Interpolation is a powerful tool in physics, allowing us to estimate unknown values between known data points. It becomes especially crucial when measurements are sparse, data is partially missing, or systems evolve too rapidly to capture every detail. Here are two striking examples from very different scales of physics where interpolation plays a central role.

Reconstructing Satellite Trajectories During Communication Blackouts

Satellites in low Earth orbit constantly send telemetry data to ground stations. However, due to signal loss during certain orbital phases—such as when passing behind the Earth or during solar storms—data gaps may occur. Suppose we know the satellite's position at two times:

- At $t_1 = 600\,\text{s}$, the position is $\vec{r}_1 = (7000, 0, 0)\,\text{km}$
- At $t_2 = 660\,\text{s}$, the position is $\vec{r}_2 = (6920, 1100, 0)\,\text{km}$

To estimate the position at $t = 630\,\text{s}$, we can apply linear interpolation:

$$\vec{r}(630) = \vec{r}_1 + \frac{30}{60}(\vec{r}_2 - \vec{r}_1) = (6960, 550, 0)\,\text{km} \tag{3.1}$$

Reconstructing Ultra-Relativistic Particle Tracks at CERN

In high-energy collisions at the Large Hadron Collider (LHC), particles are produced at nearly the speed of light and traverse multiple layers of complex detectors such as ATLAS or CMS. These detectors record "hits" where particles pass through sensor layers. However, due to noise, inefficiencies, or overlapping events (pile-up), some hits are missing.

Imagine a muon that leaves hits in 8 out of 10 tracking layers. To reconstruct its full path, we interpolate between known hits, often using spline-based or helical models. For instance:

- Known hits at $z = 0\,\text{cm}$ and $z = 10\,\text{cm}$ with $x = 0.0$ and $x = 1.2\,\text{cm}$, respectively
- Another known hit at $z = 30\,\text{cm}$, $x = 3.6\,\text{cm}$

Interpolating for the missing layer at $z = 20\,\text{cm}$ yields an estimated position of

$$x(20\,\text{cm}) \approx 2.4\,\text{cm} \tag{3.2}$$

Sophisticated filters such as the Kalman filter use this principle to estimate smooth particle trajectories, which are then used to determine momentum, charge, and even identify the particle type. Such interpolation methods are fundamental in extracting meaningful physical results from partial data — especially in the search for rare processes like Higgs boson decays or signatures of new physics.

3.2. Polynomial Interpolation

One of the most useful and well-known classes of functions mapping the set of real numbers into itself is the algebraic polynomials, the set of functions of the form

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0, \tag{3.3}$$

where n is a nonnegative integer and $a_0, \ldots, a_n$ are real constants.

One reason for the importance of polynomials is their ability to uniformly approximate continuous functions. Specifically, the Weierstrass Approximation Theorem states that for any continuous function on a closed and bounded interval, there exists a polynomial that can approximate it as closely as desired.

Weierstrass Approximation Theorem Suppose f is defined and continuous on $[a, b]$. For each $\epsilon > 0$, there exists a polynomial $P(x)$ with the property that

$$|f(x) - P(x)| < \epsilon, \quad \text{for all } x \in [a, b]. \tag{3.4}$$

Another important reason for using polynomials to approximate functions is their analytical simplicity: both the derivative and the indefinite integral of a polynomial are easy to compute and result in polynomials themselves. This makes polynomials especially convenient for approximating continuous functions.

Note on Taylor polynomials Taylor polynomials are a fundamental concept in theoretical physics and numerical analysis, but they are not commonly used for polynomial approximation over intervals. Their accuracy is concentrated near a single point x_0, as they rely solely on information at that point. As a result, they become increasingly inaccurate away from x_0, making them unsuitable for approximating functions over broader intervals. For practical computations, methods that incorporate information from multiple points are more effective. In numerical analysis, Taylor polynomials are primarily used for deriving numerical methods and estimating errors rather than for function approximation.

3.3. Lagrange Interpolating Polynomials

Determining a degree-one polynomial that passes through the distinct points (x_0, y_0) and (x_1, y_1) is equivalent to approximating a function f satisfying $f(x_0) = y_0$ and $f(x_1) = y_1$ with a first-degree polynomial. This process is known as polynomial interpolation, where the polynomial agrees with f at the given points and is used for approximation within the interval they define.

3.3.1. Construction of Lagrange polynomials for two nodes

Define the functions

$$L_0(x) = \frac{x - x_1}{x_0 - x_1} \quad \text{and} \quad L_1(x) = \frac{x - x_0}{x_1 - x_0}. \tag{3.5}$$

The linear **Lagrange interpolating polynomial** through (x_0, y_0) and (x_1, y_1) is

$$P(x) = L_0(x)f(x_0) + L_1(x)f(x_1) \tag{3.6}$$

$$= \frac{x - x_1}{x_0 - x_1}f(x_0) + \frac{x - x_0}{x_1 - x_0}f(x_1). \tag{3.7}$$

Note that

$$L_0(x_0) = 1, \quad L_0(x_1) = 0, \quad L_1(x_0) = 0, \quad \text{and} \quad L_1(x_1) = 1, \tag{3.8}$$

which implies that

$$P(x_0) = 1 \cdot f(x_0) + 0 \cdot f(x_1) = f(x_0) = y_0 \tag{3.9}$$

and

$$P(x_1) = 0 \cdot f(x_0) + 1 \cdot f(x_1) = f(x_1) = y_1. \tag{3.10}$$

So, P is the unique polynomial of degree at most one that passes through (x_0, y_0) and (x_1, y_1).

3.3.2. Extension to many nodes

To generalize the concept of linear interpolation, consider the construction of a polynomial of degree at most n that passes through the $n + 1$ points

$$(x_0, f(x_0)), \quad (x_1, f(x_1)), \quad \ldots, \quad (x_n, f(x_n)). \tag{3.11}$$

In this case, we first construct, for each $k = 0, 1, \ldots, n$, a function $L_{n,k}(x)$ with the property that $L_{n,k}(x_i) = 0$ when $i \neq k$ and $L_{n,k}(x_k) = 1$. To satisfy $L_{n,k}(x_i) = 0$ for each $i \neq k$, the numerator of $L_{n,k}(x)$ must contain the term

$$(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n). \tag{3.12}$$

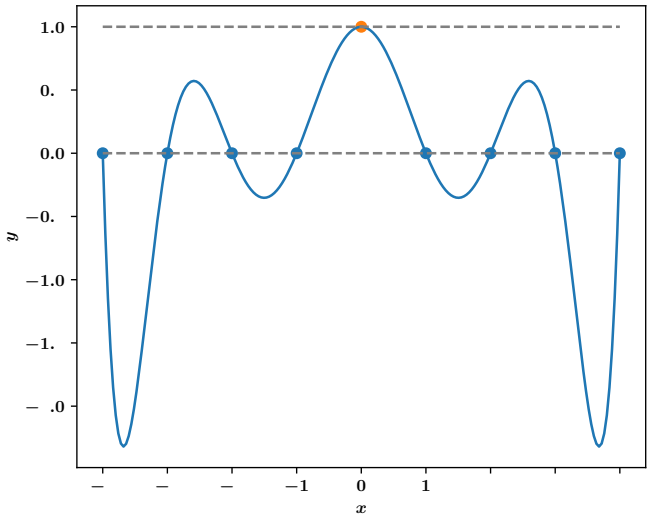

Figure 3.1.: Illustration of an even Lagrange polynomial $L_{n,k}$ for $n = 9$ and $k = 4$.

To satisfy $L_{n,k}(x_k) = 1$, the denominator of $L_{n,k}(x)$ must be this same term evaluated at $x = x_k$. Thus,

$$L_{n,k}(x) = \frac{(x - x_0) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}. \tag{3.13}$$

A sketch of the graph of a typical $L_{n,k}$ is shown in Figure 3.1

Given $n+1$ distinct points $x_0, x_1, \ldots, x_n$ and a function f defined at these points, there exists a unique polynomial $P(x)$ of degree at most n that interpolates the function—that is, it satisfies $P(x_k) = f(x_k)$ for each $k = 0, 1, \ldots, n$. This polynomial is known as the **nth Lagrange interpolating polynomial**, and it can be constructed easily once the form of the basis functions $L_{n,k}(x)$ is known. From

$$f(x_k) = P(x_k), \quad \text{for each } k = 0, 1, \ldots, n. \tag{3.14}$$

we find the polynomial by

$$P(x) = f(x_0)L_{n,0}(x) + \cdots + f(x_n)L_{n,n}(x) \tag{3.15}$$

$$= \sum_{k=0}^{n} f(x_k)L_{n,k}(x), \tag{3.16}$$

where, for each $k = 0, 1, \ldots, n$, the $L_{n,k}$ are defined as above, so

$$L_{n,k}(x) = \prod_{\substack{i=0 \\ i \neq k}}^{n} \frac{x - x_i}{x_k - x_i}. \tag{3.17}$$

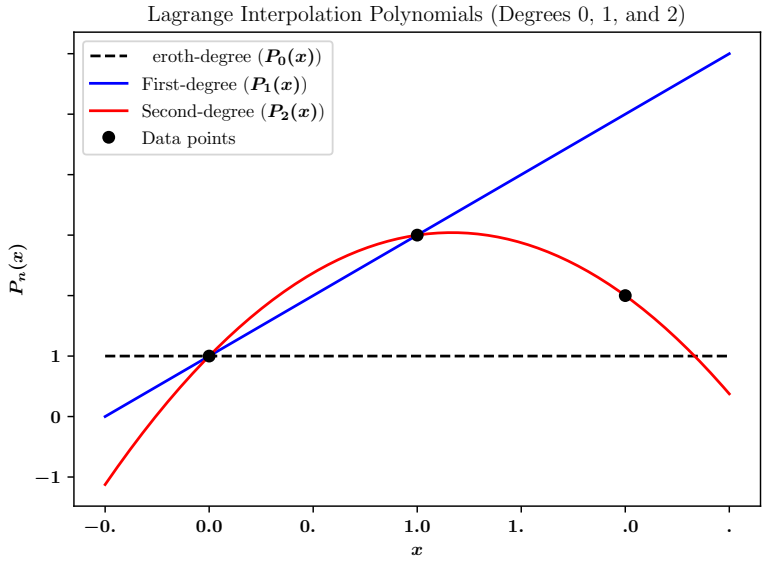

Figure 3.2.: Caption

3.3.3. Illustration of low-degree polynomials

Zeroth-degree interpolation ($n = 0$): A constant function passing through the single point (x_0, f_0).

$$P_0(x) = f_0$$

First-degree interpolation ($n = 1$): This gives a straight line passing through the two points (x_0, f_0) and (x_1, f_1).

$$P_1(x) = f_0 \cdot \frac{x - x_1}{x_0 - x_1} + f_1 \cdot \frac{x - x_0}{x_1 - x_0}$$

Second-degree interpolation ($n = 2$):

$$P_2(x) = f_0 \cdot \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + f_1 \cdot \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + f_2 \cdot \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

The three examples are illustrated in Fig. 3.2.

Example

(a) We use the nodes $x_0 = 2$, $x_1 = 2.75$, and $x_2 = 4$ to find the second Lagrange interpolating polynomial for $f(x) = 1/x$.

We first determine the coefficient polynomials $L_0(x)$, $L_1(x)$, and $L_2(x)$:

$$L_0(x) = \frac{(x-2.75)(x-4)}{(2-2.75)(2-4)} = \frac{2}{3}(x-2.75)(x-4), \tag{3.18}$$

$$L_1(x) = \frac{(x-2)(x-4)}{(2.75-2)(2.75-4)} = -\frac{16}{15}(x-2)(x-4), \tag{3.19}$$

$$L_2(x) = \frac{(x-2)(x-2.75)}{(4-2)(4-2.75)} = \frac{2}{5}(x-2)(x-2.75). \tag{3.20}$$

The function values are $f(x_0) = f(2) = \frac{1}{2}$, $f(x_1) = f(2.75) = \frac{4}{11}$, and $f(x_2) = f(4) = \frac{1}{4}$, so

$$P(x) = \sum_{k=0}^{2} f(x_k)L_k(x) \tag{3.21}$$

$$= \frac{1}{2}(x-2.75)(x-4) - \frac{4}{11}(x-2)(x-4) + \frac{1}{4}(x-2)(x-2.75) \tag{3.22}$$

which simplifies to

$$P(X) = \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}. \tag{3.23}$$

(b) Use this polynomial to approximate $f(3) = 1/3$.

An approximation to $f(3) = \frac{1}{3}$ is

$$f(3) \approx P(3) = \frac{9}{22} - \frac{105}{88} + \frac{49}{44} = \frac{29}{88} \approx 0.32955. \tag{3.24}$$

3.3.4. Error of Lagrange Polynomial Interpolation

Let f be a function that is $n+1$ times continuously differentiable on $[a,b]$, and let $P_n(x)$ be the degree-n Lagrange interpolating polynomial for f based on distinct nodes $x_0, x_1, \ldots, x_n$. Then for each $x \in [a,b]$, there exists $\xi \in [a,b]$ such that:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \cdot \omega_{n+1}(x), \tag{3.25}$$

where the **error polynomial** is defined as:

$$\omega_{n+1}(x) = \prod_{i=0}^{n}(x - x_i). \tag{3.26}$$

Implications

- The error depends on the $(n+1)$st derivative of f, and on the magnitude of the error polynomial $\omega_{n+1}(x)$.
- As n increases, the degree of ω_{n+1} increases, and its oscillation tends to grow—especially if the interpolation nodes are equally spaced.

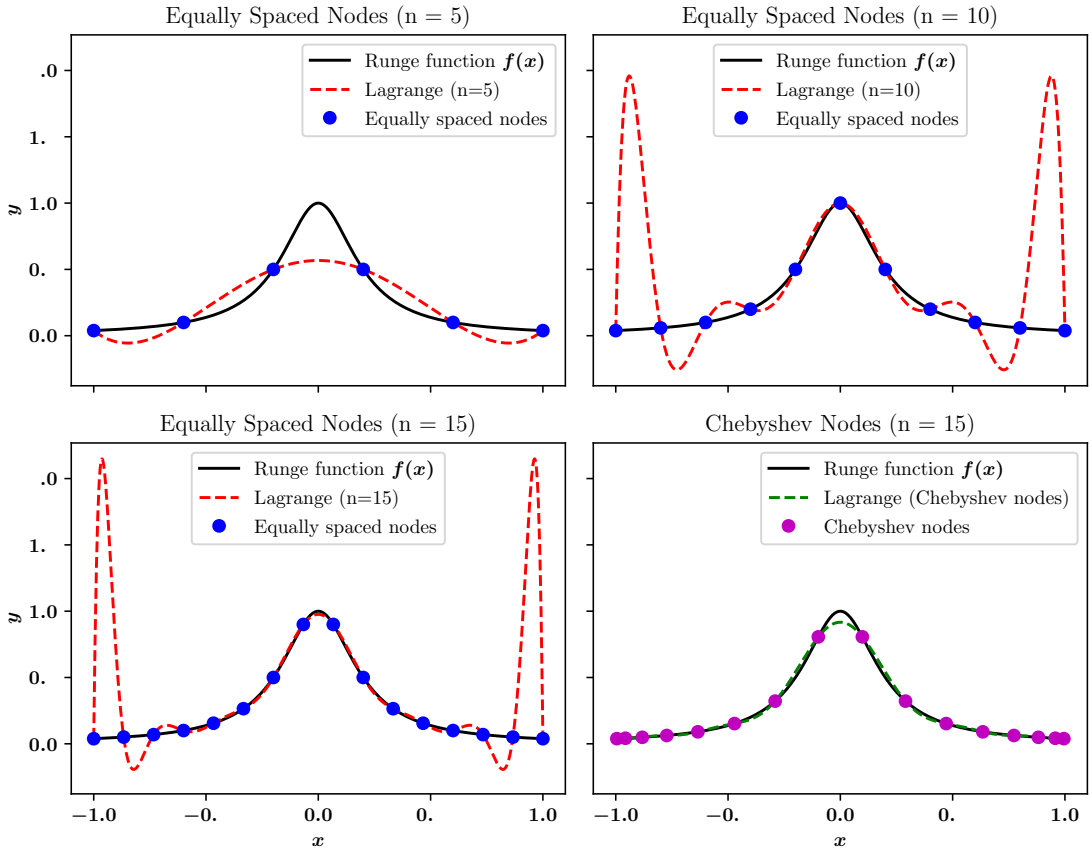

Figure 3.3.: Runge's phenomenon. Even for a smooth function, the Lagrange polynomial interpolation can cause strong oscillations. The problem can be cured by using non-equally spaced nodes, such as Chebyshev nodes.

Runge's Phenomenon

Let $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$. If we use equally spaced nodes and construct $P_n(x)$, then for large n, the interpolation error increases near the endpoints:

- The interpolating polynomial exhibits large oscillations at the edges.
- This is due to the large magnitude of $\omega_{n+1}(x)$ near the endpoints.

This phenomenon shows that high-degree Lagrange interpolation with equispaced nodes can diverge, even for smooth functions, see Fig. 3.3. We can reduce the error by

- Using non-uniform node distributions, such as Chebyshev nodes.
- Or using **piecewise interpolation methods** like splines.

3.3.5. Advantages and Disadvantages of Lagrange Polynomial Interpolation

Advantages

- **Simple construction:** No need to solve a system of equations; the interpolation formula directly uses the given data points.
- **Explicit formula:** The interpolation polynomial is given explicitly once the points are known.
- **Good for small number of points:** For small n, Lagrange interpolation provides good approximations efficiently.
- **Theoretically exact:** If the underlying function is a polynomial of degree n or less, the interpolation is exact.
- **Flexible nodes:** Arbitrary (non-equally spaced) interpolation points can be used.

Disadvantages

- **Computationally expensive for large n:** Evaluating the Lagrange polynomial becomes slow as n grows.
- **Global recalculation:** Adding or modifying a data point requires recomputing the entire polynomial.
- **Numerical instability:** For large n or widely spaced nodes, oscillations (Runge's phenomenon) can occur.
- **Poor extrapolation behavior:** The polynomial tends to behave badly outside the interpolation interval.
- **Complex expressions:** The resulting polynomials can become complicated and difficult to simplify or work with.

3.4. Spline Interpolation

3.4.1. Motivation

The previous sections concerned the approximation of arbitrary functions on closed intervals using a *single* polynomial. However, high-degree polynomials can oscillate erratically over the entire range with just minor fluctuations over a small portion of the interval.

An alternative approach is to divide the approximation interval into a collection of subintervals and construct a (generally) different approximating polynomial on each subinterval. This is called **piecewise-polynomial approximation**.

Piecewise-Polynomial Approximation

The simplest piecewise-polynomial approximation is **piecewise-linear** interpolation, which consists of joining a set of data points

$$\{(x_0, f(x_0)), (x_1, f(x_1)), \ldots, (x_n, f(x_n))\} \tag{3.27}$$

by a series of straight lines. The linear interpolation between two points $(x_i, y_i = f(x_i))$ and $(x_{i+1}, y_{i+1} = f(x_{i+1}))$ is given by:

$$f(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i) \tag{3.28}$$

for $x \in [x_i, x_{i+1}]$. A disadvantage of piecewise linear interpolation is that the function is generally not differentiable at the endpoints of subintervals.The simplest type of differentiable piecewise-polynomial function on an entire interval $[x_0, x_n]$ is obtained by fitting one quadratic polynomial between each successive pair of nodes, i.e. a parabula on $[x_0, x_1]$ agreeing with the function at x_0 and x_1, a second one on $[x_1, x_2]$ agreeing with the function at x_1 and x_2, and so on. A general quadratic polynomial

$$f(x) = ax^2 + bx + c \tag{3.29}$$

has three arbitrary constants, a, b, and c. Only two conditions are required to fit the data at the endpoints of each subinterval. The additional degree of freedom permits the can be chosen such that the interpolant has a continuous derivative on $[x_0, x_n]$.

3.4.2. Cubic Spline interpolation

Very often in physics we are not only interested in a continuous function, and its first derivative, but also higher order derivatives should be smooth. Imagine the trajectory of an object and the related equations of motion. The particle position x, its velocity $v = \dot{x}$, as well as the acceleration $a = \dot{v} = \ddot{x}$ should all behave *well*. For piecewise linear approximations for the positions x, the velocity has a jump at the intersection points, and the acceleration is infinite. A quadratic interpolation for x, which matches continuity as well as a smooth first derivative will smoothly change the velocity, but with a jump in the acceleration. A cubic interpolation will allow for a smooth description of all three quantities.

The cubic spline $S(x)$ is defined as the piecewise function between two consecutive nodes, which satisfies the following conditions:

(a) $S(x)$ is a cubic polynomial

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \tag{3.30}$$

denoted $S_j(x)$, on the subinterval $[x_j, x_{j+1}]$ for each $j = 0, 1, \ldots, n-1$;

(b) $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$ for each $j = 0, 1, \ldots, n-1$;

(c) $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$; *(Implied by (b).)*

(d) $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$;

(e) $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$ for each $j = 0, 1, \ldots, n-2$;

(f) One of the following sets of boundary conditions is satisfied:

(i) $S''(x_0) = S''(x_n) = 0$ **(natural / free boundary)**;

(ii) $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$ **(clamped boundary)**.

If free boundary conditions are applied the spline is called **natural spline**. which continues as straight lines at (and beyond) the endpoints of the interval $[a, b]$.

Construction of a Cubic Spline

As the preceding example demonstrates, a spline defined on an interval that is divided into n subintervals will require determining $4n$ constants. To construct the cubic spline interpolant for a given function f, the conditions in the definition are applied to the cubic polynomials

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3, \tag{3.31}$$

for each $j = 0, 1, \ldots, n-1$. Since $S_j(x_j) = a_j = f(x_j)$, condition (c) can be applied to obtain

$$a_{j+1} = S_j(x_{j+1}) = a_j + b_j(x_{j+1} - x_j) + c_j(x_{j+1} - x_j)^2 + d_j(x_{j+1} - x_j)^3, \tag{3.32}$$

for each $j = 0, 1, \ldots, n-2$.

The terms $x_{j+1} - x_j$ are used repeatedly in this development, so it is convenient to introduce the simpler notation

$$h_j = x_{j+1} - x_j, \tag{3.33}$$

for each $j = 0, 1, \ldots, n-1$. If we also define $a_n = f(x_n)$, then the equation

$$a_{j+1} = a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 \tag{3.34}$$

holds for each $j = 0, 1, \ldots, n-1$.

In a similar manner, define $b_n = S'(x_n)$ and observe that

$$S'_j(x) = b_j + 2c_j(x - x_j) + 3d_j(x - x_j)^2 \tag{3.35}$$

implies $S'_j(x_j) = b_j$, for each $j = 0, 1, \ldots, n-1$. Applying condition in part (d) gives

$$b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2, \tag{3.36}$$

for each $j = 0, 1, \ldots, n-1$.

Another relationship between the coefficients of S_j is obtained by defining $c_n = S''(x_n)/2$ and applying condition in part (e). Then, for each $j = 0, 1, \ldots, n-1$,

$$c_{j+1} = c_j + 3d_j h_j. \tag{3.37}$$

Solving for d_j in Eq. (3.37) and substituting this value into Eqs. (3.34) and (3.36) gives, for each $j = 0, 1, \ldots, n-1$, the new equations

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3}(2c_j + c_{j+1}) \tag{3.38}$$

and

$$b_{j+1} = b_j + h_j(c_j + c_{j+1}). \tag{3.39}$$

The final relationship involving the coefficients is obtained by solving the appropriate equation in the form of Eq. (3.38), first for b_j,

$$b_j = \frac{1}{h_j}(a_{j+1} - a_j) - \frac{h_j}{3}(2c_j + c_{j+1}), \tag{3.40}$$

and then, with a reduction of the index, for b_{j-1}. This gives

$$b_{j-1} = \frac{1}{h_{j-1}}(a_j - a_{j-1}) - \frac{h_{j-1}}{3}(2c_{j-1} + c_j). \tag{3.41}$$

Substituting these values into the equation derived from Eq. (3.39), with the index reduced by one, gives the linear system of equations

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}), \tag{3.42}$$

for each $j = 1, 2, \ldots, n-1$.

The only unknowns in the system are the c_i. The spacings h_i are given by the nodes x_i, the a_i by the function values at the nodes. Once the c_i are determined, we can compute b_i via Eq. (3.40) and then d_i using Eq. (3.37) and finally construct the spline.

Natural Splines

If f is defined at $a = x_0 < x_1 < \cdots < x_n = b$, then f has a unique natural spline interpolant S on the nodes $x_0, x_1, \ldots, x_n$; that is, a spline interpolant that satisfies the natural boundary conditions $S''(a) = 0$ and $S''(b) = 0$. The boundary conditions in this case imply that $c_n = S''(x_n)/2 = 0$ and that

$$0 = S''(x_0) = 2c_0 + 6d_0(x_0 - x_0), \tag{3.43}$$

so $c_0 = 0$.

The two equations $c_0 = 0$ and $c_n = 0$ together with the equations in (3.42) produce a linear system described by the vector equation $\mathbf{Ax} = \mathbf{b}$, where A is the $(n+1) \times (n+1)$ matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & h_2 & \cdots \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ 0 & \cdots & 0 & 0 & 1 \end{bmatrix}. \tag{3.44}$$

and $\mathbf{b}$ and $\mathbf{x}$ are the vectors

$$\mathbf{b} = \begin{bmatrix} 0 \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}. \tag{3.45}$$

Without proof: The matrix A has a unique solution for $c_0, c_1, \ldots c_n$. The solution with the boundary conditions $S''(x_0) = S''(x_n) = 0$ can be obtained by applying the following algorithm

Algorithm to compute Natural Cubic Splines

To construct the cubic spline interpolant S for the function f, defined at the numbers $x_0 < x_1 < \cdots < x_n$, satisfying $S''(x_0) = S''(x_n) = 0$:

given: n; $x_0, x_1, \ldots, x_n$; $a_0 = f(x_0)$, $a_1 = f(x_1), \ldots,$ $a_n = f(x_n)$.

output: a_j, b_j, c_j, d_j for $j = 0, 1, \ldots, n-1$.

Step 1: For $i = 0, 1, \ldots, n-1$ set $h_i = x_{i+1} - x_i$.

Step 2: For $i = 1, 2, \ldots, n-1$ set

$$\alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1}). \tag{3.46}$$

Step 3: (Steps 3, 4, and 5 and part of Step 6 solve a tridiagonal linear system)

$$\text{Set } l_0 = 1, \mu_0 = 0, \quad z_0 = 0. \tag{3.47}$$

Step 4: For $i = 1, 2, \ldots, n-1$ set

$$l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1}, \tag{3.48}$$

$$\mu_i = \frac{h_i}{l_i}, \tag{3.49}$$

$$z_i = \frac{\alpha_i - h_{i-1} z_{i-1}}{l_i}. \tag{3.50}$$

Step 5: Set $l_n = 1$, $z_n = 0$, $\mu_n = 0$.

Step 6: For $j = n-1, n-2, \ldots, 0$ set

$$c_j = z_j - \mu_j c_{j+1}, \tag{3.51}$$

$$b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j}{3}(c_{j+1} + 2c_j), \tag{3.52}$$

$$d_j = \frac{c_{j+1} - c_j}{3h_j}. \tag{3.53}$$

Step 7: OUTPUT (a_j, b_j, c_j, d_j) for $j = 0, 1, \ldots, n-1$.
STOP.

Comparison 1D interpolation methods

In Fig. 3.4 we show the comparison of the different interpolation methods.

3.5. Interpolation Along a 3D Trajectory

Suppose we are given a sequence of points in 3D space:

$$\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_n, \quad \text{where} \quad \mathbf{p}_i = (x_i, y_i, z_i). \tag{3.54}$$

We wish to interpolate a smooth curve passing through these points.

3.5.1. Parameterization

We introduce a parameter t to describe position along the curve:

- Simplest choice: assign $t_0 = 0, \quad t_n = 1$ and spread the intermediate t_i uniformly.
- Better choice: use arc-length parameterization.

In order to do this, compute the cumulative distances between all given points,

$$s_i = \sum_{k=0}^{i-1} \|\mathbf{p}_{k+1} - \mathbf{p}_k\|, \quad \text{for } i = 1, \ldots, n, \tag{3.55}$$

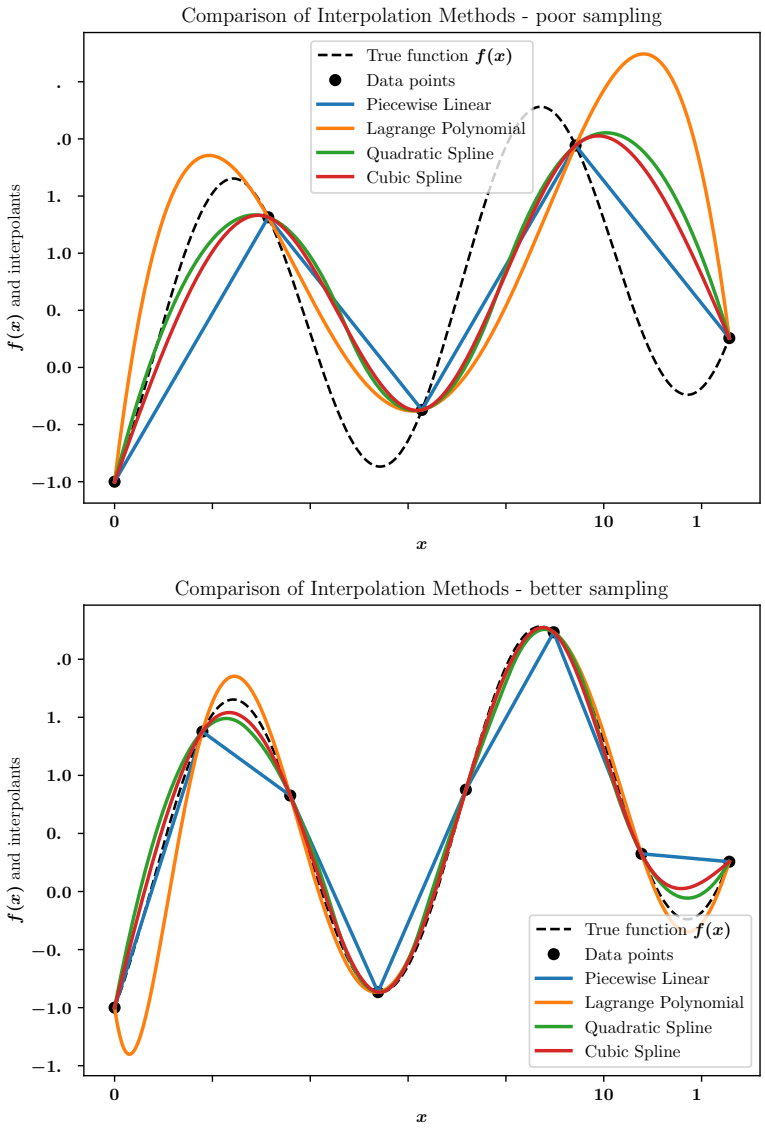

Figure 3.4.: Illustration of the different interpolation methods in 1D for two different numbers of nodes.

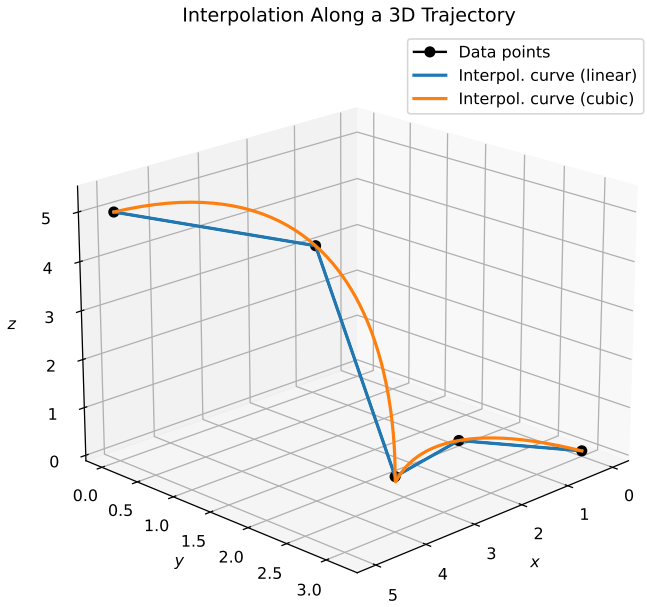

Figure 3.5.: Parameterized interpolation along a trajectory using linear and cubic interpolation.

i.e.

$$s_0 = 0 \tag{3.56}$$
$$s_1 = \|\mathbf{p}_1 - \mathbf{p}_0\| \tag{3.57}$$
$$s_2 = \|\mathbf{p}_1 - \mathbf{p}_0\| + \|\mathbf{p}_2 - \mathbf{p}_1\| \tag{3.58}$$
$$\cdots, \tag{3.59}$$

where $\|\cdot\|$ denotes the Euclidean norm. Then normalize the parameterized steps according to the length of the trajectory

$$t_i = \frac{s_i}{s_n}. \tag{3.60}$$

3.5.2. Interpolation

We interpolate the coordinates separately as functions of t:

$$x(t), \quad y(t), \quad z(t), \tag{3.61}$$

using 1D interpolation methods such as piecewise linear interpolation, cubic splines, or higher-order techniques. The resulting curve is then given by

$$\mathbf{p}(t) = (x(t), y(t), z(t)), \tag{3.62}$$

an illustration is shown in Fig. 3.5.

3.5.3. Example

Given:

$$\mathbf{p}_0 = (0,0,0), \quad \mathbf{p}_1 = (1,2,0), \quad \mathbf{p}_2 = (3,3,1), \tag{3.63}$$

compute distances:

$$d_0 = \sqrt{(1-0)^2 + (2-0)^2 + (0-0)^2} = \sqrt{5}, \tag{3.64}$$

$$d_1 = \sqrt{(3-1)^2 + (3-2)^2 + (1-0)^2} = \sqrt{6}. \tag{3.65}$$

Then cumulative distances:

$$s_0 = 0, \quad s_1 = \sqrt{5}, \quad s_2 = \sqrt{5} + \sqrt{6}. \tag{3.66}$$

Normalize:

$$t_0 = 0, \quad t_1 = \frac{\sqrt{5}}{\sqrt{5} + \sqrt{6}}, \quad t_2 = 1. \tag{3.67}$$

Finally interpolate $x(t)$, $y(t)$, and $z(t)$ separately to reconstruct the 3D curve.

3.6. Multidimensional Interpolation: 2D Case

3.6.1. From 1D to 2D Interpolation

In one dimension, linear interpolation finds an intermediate value between two known points. Extending this idea to two dimensions, we interpolate first along one axis (for example, x), and then along the other axis (y). Suppose we use linear interpolation in each dimension. This sequential interpolation approach is called **bilinear interpolation** and is the simplest form of 2D interpolation.

3.6.2. Mathematical Description

Assume we know the values of a function f at the four corners of a rectangle:

$$(x_0, y_0), \quad (x_1, y_0), \quad (x_0, y_1), \quad (x_1, y_1). \tag{3.68}$$

We wish to interpolate $f(x, y)$ at an arbitrary point (x, y) inside the rectangle. The procedure is as follows (see Fig. 3.6 for an illustration)

- **First interpolation in x at fixed y_0:**

$$f(x, y_0) \approx \frac{x_1 - x}{x_1 - x_0} f(x_0, y_0) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_0), \tag{3.69}$$

  and similarly at fixed y_1:

$$f(x, y_1) \approx \frac{x_1 - x}{x_1 - x_0} f(x_0, y_1) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_1). \tag{3.70}$$

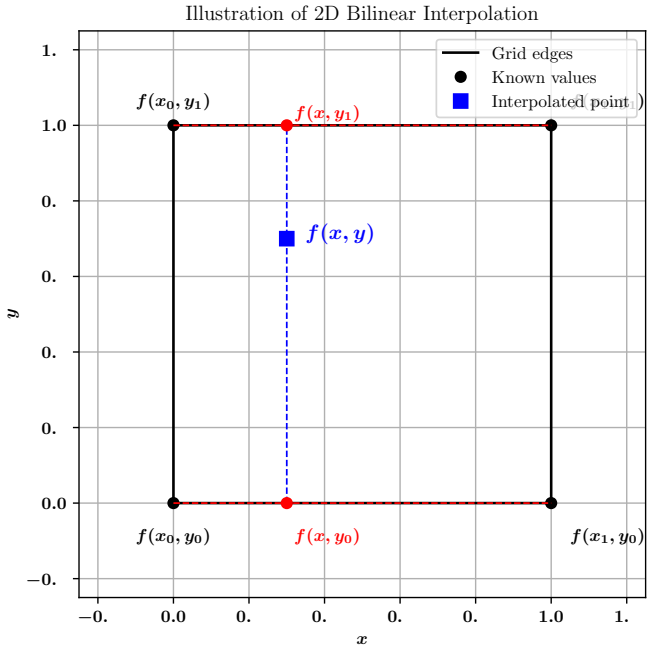

Figure 3.6.: Illustration of a 2D bilinear interpolation.

- **Then interpolation in y between the intermediate results:**

$$f(x,y) \approx \frac{y_1 - y}{y_1 - y_0} f(x, y_0) + \frac{y - y_0}{y_1 - y_0} f(x, y_1). \tag{3.71}$$

Thus, bilinear interpolation consists of two successive 1D interpolations.

3.6.3. Order of Bilinear Interpolation: Does it Matter?

In this case we would like to prove that the order does not matter. Let's write down the equations in full to see this

First interpolating along x, then y

$$\begin{aligned}
f(x,y_0) &= \frac{x_1 - x}{x_1 - x_0} f(x_0, y_0) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_0), \\
f(x,y_1) &= \frac{x_1 - x}{x_1 - x_0} f(x_0, y_1) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_1), \\
f(x,y) &= \frac{y_1 - y}{y_1 - y_0} f(x, y_0) + \frac{y - y_0}{y_1 - y_0} f(x, y_1).
\end{aligned}$$

First interpolating along y, then x

$$f(x_0, y) = \frac{y_1 - y}{y_1 - y_0} f(x_0, y_0) + \frac{y - y_0}{y_1 - y_0} f(x_0, y_1),$$
$$f(x_1, y) = \frac{y_1 - y}{y_1 - y_0} f(x_1, y_0) + \frac{y - y_0}{y_1 - y_0} f(x_1, y_1),$$
$$f(x, y) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y) + \frac{x - x_0}{x_1 - x_0} f(x_1, y).$$

Proof of equivalence We expand the full four terms:

$$f(x, y) = \frac{(x_1 - x)(y_1 - y)}{(x_1 - x_0)(y_1 - y_0)} f(x_0, y_0) \tag{3.72}$$
$$+ \frac{(x_1 - x)(y - y_0)}{(x_1 - x_0)(y_1 - y_0)} f(x_0, y_1) \tag{3.73}$$
$$+ \frac{(x - x_0)(y_1 - y)}{(x_1 - x_0)(y_1 - y_0)} f(x_1, y_0) \tag{3.74}$$
$$+ \frac{(x - x_0)(y - y_0)}{(x_1 - x_0)(y_1 - y_0)} f(x_1, y_1). \tag{3.75}$$

Thus, the result is the same regardless of whether we interpolate first in x or first in y.

Simplified Bilinear Interpolation Using Scaled Coordinates Assume a *square* grid where $x_1 - x_0 = y_1 - y_0 = h$. Define scaled coordinates:

$$\xi = \frac{x - x_0}{h}, \quad \eta = \frac{y - y_0}{h}, \quad \text{so that } \xi, \eta \in [0, 1].$$

Let the function values at the four corners be:

$$f_{00} = f(x_0, y_0), \quad f_{10} = f(x_1, y_0), \quad f_{01} = f(x_0, y_1), \quad f_{11} = f(x_1, y_1).$$

Then the bilinear interpolant becomes:

$$f(x, y) = (1 - \xi)(1 - \eta) f_{00} + \xi(1 - \eta) f_{10} + (1 - \xi)\eta f_{01} + \xi\eta f_{11}.$$

This expression is symmetric in ξ and η and shows that the result is independent of the order of interpolation.

3.6.4. Higher-Order Interpolation and Order Dependence

Multilinear Interpolation is Order-Independent In bilinear or trilinear interpolation, the interpolated value is computed as a weighted sum of corner values using linear weights. For example, in trilinear interpolation, we interpolate over a cube defined by eight corner values $f_{ijk} = f(x_i, y_j, z_k)$, where $i, j, k \in \{0, 1\}$.

With normalized coordinates:

$$\xi = \frac{x - x_0}{x_1 - x_0}, \quad \eta = \frac{y - y_0}{y_1 - y_0}, \quad \zeta = \frac{z - z_0}{z_1 - z_0},$$

the trilinear interpolation formula becomes:

$$f(x,y,z) = \sum_{i=0}^{1}\sum_{j=0}^{1}\sum_{k=0}^{1}(1-\xi)^{1-i}\xi^i \cdot (1-\eta)^{1-j}\eta^j \cdot (1-\zeta)^{1-k}\zeta^k \cdot f_{ijk}.$$

This expression is fully symmetric in all three variables and independent of the order in which the interpolation is performed. That is, whether we interpolate in x, then y, then z, or in any other order, the final result is the same.

Higher-Order Interpolation Can Be Order-Dependent Higher-order interpolation methods (e.g., bicubic or spline-based) often use not only function values but also derivatives or second derivatives. These methods yield more accurate and smoother results, but can introduce asymmetries if interpolation is performed sequentially along coordinate directions. In particular, if higher order interpolation is performed via two successive 1D interpolations (e.g., cubic spline in x, then in y), the result can vary depending on the order of application:

$$f_{x\to y}(x,y) \neq f_{y\to x}(x,y),$$

3.6.5. Symmetric Construction: Tensor-Product Bases

To avoid order dependence, higher-dimensional interpolation should be based on a symmetric construction. This is achieved by using tensor-product basis functions, e.g.,

$$P(x,y) = \sum_{i=0}^{3}\sum_{j=0}^{3} a_{ij}\phi_i(x)\psi_j(y),$$

where ϕ_i and ψ_j are 1D basis functions (e.g., B-splines or Hermite functions). In this case, the interpolated result is independent of the order in which variables are processed.

This approach is used in software libraries such as `scipy.interpolate.RectBivariateSpline`, where the interpolation surface is constructed globally from tensor products of basis functions.

3.6.6. General Polynomial Interpolant

Consider an interpolant of the form:

$$S(x,y) = \sum_i \sum_j a_{ij}\, x^i\, y^j$$

This is a **bivariate polynomial** expressed as a double sum over powers of x and y.

Is the interpolation order relevant?

Let us examine whether evaluating this interpolant by first fixing y and interpolating in x, then vice versa, would give different results.

Fixing y first:

$$S(x,y) = \sum_i \left(\sum_j a_{ij}\, y^j \right) x^i = \sum_i c_i(y)\, x^i$$

where

$$c_i(y) = \sum_j a_{ij}\, y^j$$

This effectively turns the expression into a 1D polynomial in x with y-dependent coefficients.

Fixing x first:

$$S(x,y) = \sum_j \left(\sum_i a_{ij}\, x^i \right) y^j = \sum_j d_j(x)\, y^j$$

where

$$d_j(x) = \sum_i a_{ij}\, x^i$$

Conclusion: The result does not depend on the order. Whether you compute the x-polynomial first or the y-polynomial first, you always get the same value of $S(x,y)$ because the operations are just regroupings of the same sums and multiplications (which commute).

Why is this the case?

- Polynomial basis functions (like $x^i y^j$) are **separable** (written as products of x-only and y-only powers) and **commutative** under multiplication.
- Tensor-product constructions with polynomials naturally exhibit **order independence**, just like tensor-product B-splines.

Final Conclusion: For tensor-product polynomial interpolants of the form:

$$S(x,y) = \sum_i \sum_j a_{ij}\, x^i\, y^j$$

the interpolation order does not matter — you will always obtain the same result.

But — can bicubic interpolation as a procedure be order-dependent? While the bicubic polynomial itself, is a tensor-product polynomial that is separable and order-independent, *order dependence can arise in practical implementations* due to the following factors:

1. Determination of the coefficients a_{ij}: In standard bicubic interpolation, the coefficients are determined using function values, first derivatives ($\partial f/\partial x$, $\partial f/\partial y$), and the mixed derivative ($\partial^2 f/\partial x \partial y$) at grid points. If the derivative estimates are computed using different methods along the x and y axes (e.g., central differences along x and forward differences along y), the resulting interpolant can reflect directional biases, introducing order dependence.

2. Axis-by-axis interpolation approximation: Some numerical implementations approximate bicubic interpolation by performing:

- cubic interpolation along x,
- followed by cubic interpolation along y (or vice versa).

This sequential, axis-by-axis cubic interpolation is not equivalent to fitting the full bicubic polynomial and can be order-dependent, especially when interpolating non-separable functions.

3. Data irregularities or non-smoothness: If the underlying data is noisy or has discontinuities, different derivative estimation methods or interpolation orders can lead to varying results, even though the mathematical form of $P(x, y)$ remains order-independent.

Summary Table:

Aspect	Order-dependent?
Bicubic polynomial formula	No
Derivative estimation method	Possibly yes
Sequential axis-by-axis cubic interpolation	Yes (for non-separable data)
Numerical rounding / discretization	Minor order effects possible

3.6.7. Example: B-splines

B-splines in 1D

The B-spline basis functions $N_{i,p}(x)$ of degree p are defined recursively:

Degree 0 (piecewise constant):

$$N_{i,0}(x) = \begin{cases} 1, & \text{if } t_i \leq x < t_{i+1} \\ 0, & \text{otherwise} \end{cases}$$

Degree p (recursive definition):

$$N_{i,p}(x) = \frac{x - t_i}{t_{i+p} - t_i} N_{i,p-1}(x) + \frac{t_{i+p+1} - x}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(x)$$

where $\{t_i\}$ are the knot positions. An illustration of B-spline basis functions is shown in Fig 3.7.

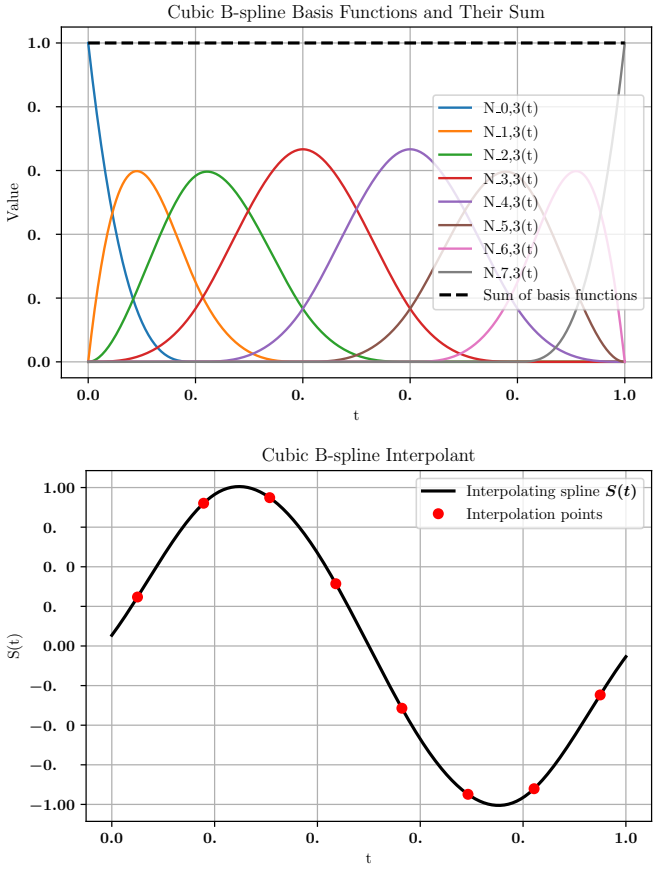

Figure 3.7.: Illustration of B-spline basis functions (top). Notice how each is non-zero only over a limited part of the domain (local support). The bottom panel shows the interpolant passing through the function values.

Interpolant: The interpolant has the form

$$S(x) = \sum_i c_i \, N_{i,p}(x),$$

where the coefficients c_i are the knot points. For our purpose of interpolation through data points, these c_i are automatically your function values f_i. Note, however, that there are plenty of other applications for B-spluines such as approximation of values, where the knot points are not fixed function values, but rather results of fitting through many data points.

Extension to 2D: Tensor-product B-splines

In 2D, the tensor-product B-spline basis functions are:

$$S(x, y) = \sum_i \sum_j c_{ij} \, N_{i,p}(x) \, M_{j,q}(y)$$

where $N_{i,p}(x)$ and $M_{j,q}(y)$ are the B-spline basis functions in the x and y-direction, respectively. The interpolation is **order-independent** due to the separable structure of the basis:

$$S(x, y) = \sum_i \left[N_{i,p}(x) \sum_j c_{ij} M_{j,q}(y) \right] \tag{3.76}$$

$$= \sum_j \left[M_{j,q}(y) \sum_i c_{ij} N_{i,p}(x) \right] \tag{3.77}$$

3.6.8. Non-separable Function Example

A simple non-separable function:

$$S(x, y) = x \cdot y + \sin(x + y)$$

The $\sin(x + y)$ term couples x and y in a way that cannot be expressed as a product of separate functions of x and y. When performing interpolation axis by axis, the order of interpolation will affect the final result.

3.6.9. Order Dependence in Multidimensional Interpolation

In multidimensional interpolation, the order in which interpolation is performed can affect the final result, depending on the method used.

Key Principles:

- **Multilinear interpolation** (e.g., bilinear, trilinear) is inherently *order-independent* because it uses symmetric linear weights applied in each dimension.
- **Tensor-product interpolants** (e.g., bicubic splines, B-splines constructed globally) preserve *order independence*, since the basis functions are separable and applied symmetrically across all dimensions.
- **Sequential 1D interpolation methods** (e.g., applying cubic splines first along x, then along y) can introduce *order dependence*, especially for non-separable functions, non-uniform grids, or when derivative estimates differ between directions.
- **Global interpolation methods** (e.g., radial basis functions, global polynomial fits) are *fully symmetric* and therefore order-independent.

3.7. Scattered Data Interpolation: Radial Basis Functions (RBF)

Given a set of scattered data points

$$\{(\mathbf{x}_i, f_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d,$$

we seek a smooth interpolant $s(\mathbf{x})$ such that

$$s(\mathbf{x}_i) = f_i \quad \text{for } i = 1, \ldots, N.$$

The Radial Basis Function (RBF) interpolant is defined as:

$$s(\mathbf{x}) = \sum_{i=1}^{N} c_i \, \phi\left(\|\mathbf{x} - \mathbf{x}_i\|\right),$$

where:

- $\phi(r)$ is a radially symmetric basis function,
- c_i are coefficients to be determined,
- $\|\mathbf{x} - \mathbf{x}_i\|$ is the Euclidean distance between $\mathbf{x}$ and $\mathbf{x}_i$.

Common Choices for $\phi(r)$

Function Type	**Formula for $\phi(r)$**
Gaussian	$e^{-(\epsilon r)^2}$
Multiquadric	$\sqrt{1 + (\epsilon r)^2}$
Inverse Multiquadric	$\frac{1}{\sqrt{1+(\epsilon r)^2}}$
Thin Plate Spline	$r^2 \log(r)$
Linear	r

Here, ϵ is a shape parameter controlling the spread of the basis functions.

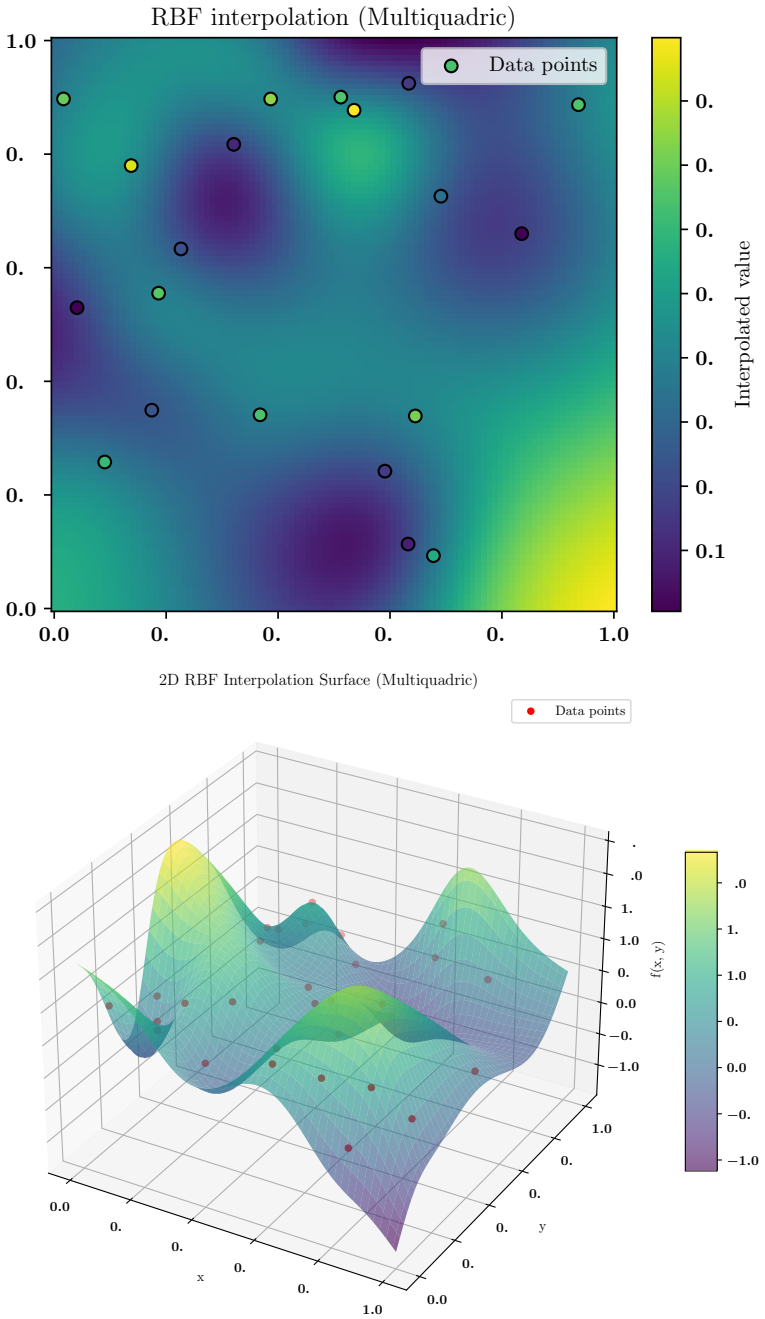

Figure 3.8.: Example of RBF interpolation. Shown are the randomly spaced points in 2D (top) with their function values in color. The bottom part visualises the function value in 3D.

Computing the Coefficients

The interpolation conditions require:

$$s(\mathbf{x}_j) = f_j, \quad \forall j = 1, \ldots, N.$$

This yields the linear system:

$$\Phi \mathbf{c} = \mathbf{f},$$

where:

$$\Phi_{ij} = \phi\left(\|\mathbf{x}_i - \mathbf{x}_j\|\right), \quad \mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{bmatrix}.$$

The coefficients $\mathbf{c}$ are obtained by solving:

$$\mathbf{c} = \Phi^{-1}\mathbf{f}.$$

Evaluation of the Interpolant

For any new point $\mathbf{x}$, the interpolant is evaluated as:

$$s(\mathbf{x}) = \sum_{i=1}^{N} c_i \, \phi\left(\|\mathbf{x} - \mathbf{x}_i\|\right).$$

We show an illustration of the points in 2D and the interpolant in 3D in Fig. 3.8.

Pros and Cons

- **Advantages:** Highly flexible, dimension-independent, handles scattered data naturally, produces smooth interpolants.
- **Disadvantages:** Computationally expensive for large N due to the cost of solving a dense linear system.

3.8. Summary of Common Methods:

Method	**Order-Independent**	**Remarks**
Bilinear / Trilinear	Yes	Linear weights, symmetric
Bicubic (tensor-product)	Yes	Symmetric basis functions
Bicubic (successive 1D splines)	No	Asymmetric intermediate steps
2D Cubic Hermite via 1D splines	No	Cross-derivative mismatch
B-spline (global grid)	Yes	Tensor-product formulation
Radial Basis Functions	Yes	Global, fully symmetric

Conclusion: For accurate and order-independent interpolation in multiple dimensions, it is recommended to use methods based on tensor-product basis functions or global interpolants. If dimension-by-dimension interpolation is applied, be aware that the interpolation result may vary with the order, particularly for higher-order methods or non-separable data.

A. Physics

A.1. Response Functions: Heat Capacity and Susceptibility

A.1.1. What Are Response Functions?

In thermodynamics and statistical physics, a **response function** tells us how a macroscopic observable reacts to a small change in a control parameter:

- The **heat capacity** C_V measures how strongly the energy responds to a change in temperature:

  $$C_V = \left(\frac{\partial \langle E \rangle}{\partial T} \right)_V$$

  Illustratively: How much does the energy fluctuate as we heat the system up?

- The **magnetic susceptibility** χ measures how strongly the magnetisation responds to a magnetic field:

  $$\chi = \left(\frac{\partial \langle M \rangle}{\partial h} \right)_T$$

  Illustratively: How easily can we magnetise the system with a weak field?

Surprisingly, these quantities can be computed *without changing* T or h, by observing the *natural fluctuations* in equilibrium.

A.1.2. Canonical Ensemble

Let the system be in contact with a heat bath at fixed temperature T. The probability of microstate i is:

$$P_i = \frac{e^{-\beta E_i}}{Z}, \qquad \beta = \frac{1}{k_\mathrm{B} T}, \qquad Z = \sum_i e^{-\beta E_i}$$

A.1.3. Heat Capacity from Energy Fluctuations

The average energy is:

$$\langle E \rangle = \sum_i E_i P_i = \frac{1}{Z} \sum_i E_i e^{-\beta E_i}$$

Differentiate with respect to T, using $\frac{d}{dT} = -\frac{1}{k_B T^2}\frac{d}{d\beta}$:

$$C_V = \frac{d\langle E\rangle}{dT} = -\frac{1}{k_B T^2}\cdot\frac{d\langle E\rangle}{d\beta}$$

Now compute:

$$\frac{d\langle E\rangle}{d\beta} = \frac{d}{d\beta}\left(\frac{1}{Z}\sum_i E_i e^{-\beta E_i}\right) = -\langle E^2\rangle + \langle E\rangle^2 = -\mathrm{Var}(E)$$

So:

$$\boxed{C_V = \frac{1}{k_B T^2}\left(\langle E^2\rangle - \langle E\rangle^2\right) = k_B\beta^2\left(\langle E^2\rangle - \langle E\rangle^2\right)}$$

A.1.4. Magnetic Susceptibility from Magnetisation Fluctuations

Introduce a magnetic field h into the Hamiltonian:

$$\mathcal{H} = \mathcal{H}_0 - hM$$

Then the probability becomes:

$$P_i = \frac{e^{-\beta(E_i - hM_i)}}{Z}$$

The average magnetisation is:

$$\langle M\rangle = \sum_i M_i P_i = \frac{1}{Z}\sum_i M_i e^{-\beta(E_i - hM_i)} = \frac{\partial}{\partial(\beta h)}\ln Z$$

Differentiate with respect to h:

$$\chi = \frac{\partial\langle M\rangle}{\partial h} = \frac{\partial^2 \ln Z}{\partial(\beta h)^2}\cdot\left(\frac{\partial(\beta h)}{\partial h}\right)^2 = \beta\left(\langle M^2\rangle - \langle M\rangle^2\right)$$

Hence:

$$\boxed{\chi = \frac{1}{k_B T}\left(\langle M^2\rangle - \langle M\rangle^2\right) = \beta\left(\langle M^2\rangle - \langle M\rangle^2\right)}$$

This expression is rigorous in the canonical ensemble. In computational models where $k_B = 1$, it simplifies to $\chi = \beta\,\mathrm{Var}(M)$.

A.1.5. Interpretation

- The response functions C_V and χ are directly related to *fluctuations* of their conjugate variables (energy and magnetisation).
- These are exact results for equilibrium systems in the canonical ensemble.
- Near a phase transition, these fluctuations often diverge in the thermodynamic limit, making C_V and χ useful diagnostics.